Solution: Frequency Sort

Problem Statement

Given a string, sort it based on the decreasing frequency of its characters.

Example 1:

```
Input: "Programming"
Output: "rrggmmPiano"
Explanation: 'r', 'g', and 'm' appeared twice, so they need to appear before any other character.
```

Example 2:

```
Input: "abcbab"
Output: "bbbaac"
Explanation: 'b' appeared three times, 'a' appeared twice, and 'c' appeared only once.
```

Solution

This problem follows the **Top 'K' Elements** pattern, and shares similarities with `Top 'K' Frequent Numbers`.

We can follow the same approach as discussed in the `Top 'K' Frequent Numbers` problem. First, we will find the frequencies of all characters, then use a max-heap to find the most occurring characters.

Code

Here is what our algorithm will look like:

Java

```
import java.util.*;

class Solution {

  public String sortCharacterByFrequency(String str) {
    // find the frequency of each character
    Map<Character, Integer> characterFrequencyMap = new HashMap<>();
```

```
8       for (char chr : str.toCharArray()) {
9         characterFrequencyMap.put(chr, characterFrequencyMap.getOrDefault(chr, 0) + 1);
10      }
11
12      PriorityQueue<Map.Entry<Character, Integer>> maxHeap =
13                      new PriorityQueue<Map.Entry<Character, Integer>>(
14                                      (e1, e2) -> e2.getValue() - e1.getValue());
15
16      // add all characters to the max heap
17      maxHeap.addAll(characterFrequencyMap.entrySet());
18
19      // build a string, appending the most occurring characters first
20      StringBuilder sortedString = new StringBuilder(str.length());
21      while (!maxHeap.isEmpty()) {
22        Map.Entry<Character, Integer> entry = maxHeap.poll();
23        for (int i = 0; i < entry.getValue(); i++)
24          sortedString.append(entry.getKey());
```

Run Code Result | Stdout

You must run your code first

Execute

Time Complexity

The time complexity of the above algorithm is O(D∗logD) where ‘D’ is the number of distinct characters in the input string. This means, in the worst case, when all characters are unique the time complexity of the algorithm will be O(N∗logN) where ‘N’ is the total number of characters in the string.

Space Complexity

The space complexity will be O(N), as in the worst case, we need to store all the ‘N’ characters in the HashMap.

← Previous

Frequency Sort (Medium)

Next →

Kth Largest Number In A Stream (Medium)

Mark as Completed